

Reviewer Pack — Hyperbolic Geometry of Boolean Functions Bounds Communicatio...

Entry 5b51b919151a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 18:42:26 UTC

Hyperbolic Geometry of Boolean Functions Bounds Communication Complexity

Entry ID: 5b51b919151a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 18:42:26 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic geometry **Field B** (complexity object): Communication complexity

Statement:

For every $N \times N$ Boolean matrix M , the hyperbolic metric diameter $d_H(M)$ is lower-bounded by a constant $c \cdot CC_R(M)$, where $CC_R(M)$ is the randomized two-party communication complexity of M . Equivalently: for all $N \leq 40$ and for every set of 30 random seeds, there exists a constant $c > 0$ such that $d_H(M) \geq c \cdot CC_R(M)$.

Rationale (proposer's reasoning):

Hyperbolic geometry has been used to study discrete structures, and its negative curvature might relate to the complexity of communication tasks. This conjecture suggests a potential connection between geometric properties and communication complexity, which could lead to new insights into lower bounds.

Taxonomy category: hyperbolic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f15a68bbc313a843

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient (ρ) between $d_H(M)$ and $CC_R(M)$ for all generated $N \times N$ Boolean matrices M with $N \leq 40$ and 30 random seeds is greater than or equal to a threshold of 0.8, AND the mean ρ does not exceed 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:(Hyperbolic geometry AND Communication complexity) AND (Boolean functions OR communication complexity) - title:(Hyperbolic metric diameter AND Boolean matrix) AND (communication complexity OR randomized two-party communication complexity) - subject:(Hyperbolic geometry in Boolean functions) AND (bounds on communication complexity) AND (randomized two-party communication complexity)

Top relevant hits considered: - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/cs/0608054v2] Dispersion of Mass and the Complexity of Randomized Geometric Algorithms - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1403.8100] Recent advances on the log-rank conjecture in communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_matrix(N):
    return [[random.choice([0, 1]) for _ in range(N)] for _ in range(N)]

def hyperbolic_metric_diameter(M):
    N = len(M)
    max_dist = 0
    for i in range(N):
        for j in range(i + 1, N):
            dist = sum(abs(M[i][k] - M[j][k]) for k in range(N))
            if dist > max_dist:
                max_dist = dist
    return max_dist

def randomized_two_party_communication_complexity(M):
    N = len(M)
    # Simplified model: each bit requires 1 unit of communication
    return sum(sum(row) for row in M)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

results = []
for n in [5, 10, 15, 20, 30, 40]:
    N = n
    M = generate_random_matrix(N)
    d_H = hyperbolic_metric_diameter(M)
    CC_R = randomized_two_party_communication_complexity(M)
    results.append((d_H, CC_R))

if not results:
    return {
        "metric_name": "Spearman rank correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

# Calculate Spearman rank correlation coefficient
ranks_d_H = {d_H: i for i, (d_H, _) in enumerate(sorted(set(d_H for d_H, _ in results)), start=0)}
ranks_CC_R = {CC_R: i for i, (_, CC_R) in enumerate(sorted(set(CC_R for _, CC_R in results)), start=0)}

n = len(results)
rho_numerator = sum((ranks_d_H[d_H] - ranks_CC_R[CC_R]) ** 2 for d_H, CC_R in results)
rho_denominator = (n * (n**2 - 1)) / 12
rho = 1 - (6 * rho_numerator) / rho_denominator

return {
    "metric_name": "Spearman rank correlation coefficient",
    "metric_value": rho,
    "instances_tested": n,
    "n_max": max(n for _, n in results),
    "conjecture_holds": rho >= 0.8 and rho <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"n\": {trial_result['n_max']}}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_rho = sum(result["metric_value"] for result in results) / len(results)
        std_rho = math.sqrt(sum((result["metric_value"] - mean_rho) ** 2 for result in results) / len(results))
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Spearman rank correlation coefficient does not hold for seed={first_failing_seed}\"")
    else:
        print("RESULT: INCONCLUSIVE no trials supported")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7a6239b.py", line 79, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7a6239b.py", line 57, in run_trial
    ranks_d_H = {d_H: i for i, (d_H, _) in enumerate(sorted(set(d_H for d_H, _ in results)), start=1)}
                   ~~~~~^
TypeError: cannot unpack non-iterable int object
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22086
2	preregistration	ollama_remote	glm4:latest	0	0	23165
3	novelty	ollama_remote	glm4:latest	0	0	16048
4	novelty	ollama_remote	glm4:latest	0	0	16634
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21809
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10910
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20510
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12119
9	judge	ollama_remote	glm4:latest	0	0	8936

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152216 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5b51b919151a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5b51b919151a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5b51b919151a.tar.gz` (if generated)